
NATIONAL ECONOMICS UNIVERSITY (NEU)
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL SOLUTIONS & MARKING RUBRIC

Course: Fundamental Programming Concepts in Python (DSAI1003)

Progress Exam: Functions, Modular Design and Variable Scope in Python

Note for Instructors and Teaching Assistants: This document provides complete answer keys, detailed rationale for conceptual questions, exact output traces, step-by-step modular program design, complete reference implementations, and an itemized marking rubric designed for rigorous and consistent grading across all student cohorts.

Section A: Functions, Modular Design & Scope Concepts (20 points)

2 points per question. Total = 20 points.

The comprehensive answer key and rationale table for Section A begins on the following page.

Q#	Key	Conceptual Explanation & Rule Reference	Marks
Q1	B	The black-box abstraction specifies that a caller only interacts with a function through its documented interface (parameters, return types, pre/post-conditions). The internal implementation details are hidden and can be refactored without breaking the caller.	2 pts
Q2	C	A parameter is a variable declared in the function definition header (def) that serves as a placeholder. An argument is the actual value or expression evaluated and passed into the function when it is invoked.	2 pts
Q3	B	The return statement passes computed data back to the calling expression for storage, branching, or further math. In contrast, print() is an I/O side effect that writes text to standard output and returns None.	2 pts
Q4	C	In Python, every function returns a value. If execution reaches the end of the body without encountering a return statement, or executes a bare return, Python implicitly returns the special singleton value None of type NoneType.	2 pts
Q5	B	Parameter variables are local variables initialized with the argument values. In Python, reassigning a parameter variable (x = x + 10) merely points the local name x to a new integer object, leaving the caller's variable a completely unchanged at 5.	2 pts
Q6	B	A local variable has local scope and function-duration lifetime: it is allocated upon function entry, exists only during execution of the function call frame, is destroyed upon return, and is strictly inaccessible outside the function.	2 pts
Q7	C	When a local variable inside a function shares the name of a global variable, the local variable shadows (masks) the global variable. Inside that function, references to the name resolve to the local variable.	2 pts
Q8	B	Stepwise refinement (top-down design) is the software engineering practice of decomposing a large, complex problem into smaller, intellectually manageable, and independently testable sub-tasks, each implemented as a dedicated helper function.	2 pts
Q9	B	A stub is a minimal placeholder function with a valid signature and return statement (returning a safe dummy value such as 0, False, or "") used during incremental top-down development to test calling structures before full implementation.	2 pts
Q10	C	Encapsulating the main application logic inside a def main(): function avoids polluting the global namespace with script-level variables, provides an explicit execution entry point, and enables modularity and automated unit testing.	2 pts

Section B: Output Tracing, Scope Diagnosis & Error Correction (25 points)

Questions 11 through 15 evaluate concrete understanding of call semantics, variable scope, mutation isolation, syntax defects, and formal specification. 5 points each. Total = 25 points.

Q11. Exact Output Tracing with Parameter Passing & Return Tuples (5 points)

Solution for Q11

```
Subtotal: $100.00 | Discount: $15.00
Final Due: $93.00
Saved: True
```

Scoring Breakdown:

- **Line 1 (2 points):** Subtotal: \$100.00 | Discount: \$15.00 (*total* = 100.0 remains unchanged; *is_member* = *True* → *rate* = 0.15; *compute_discount*(100.0, 0.15) returns 15.0).
- **Line 2 (1.5 points):** Final Due: \$93.00 (*tax* = $100.0 \times 0.08 = 8.0$; *net_total* = $100.0 - 15.0 + 8.0 = 93.0$, formatted as \$93.00).
- **Line 3 (1.5 points):** Saved: True ($15.0 \geq 15.0$ is True and $100.0 == 100.0$ is True → True and True evaluates to True).

Q12. Evaluation of Function Calls and Return Values (5 points)

Solution for Q12

- **a) notify("Done")** is None: **True** (1 pt) – Function has no return statement, so it returns None; None is None evaluates to True.
- **b) cap(20) + cap(60): 90** (1 pt) – *cap*(20) = $20 \times 2 = 40$ (since $20 \leq 50$); *cap*(60) = 50 (since $60 > 50$); $40 + 50 = 90$.
- **c) valid("NE2026"):** **True** (1 pt) – Length is 6, "NE".isalpha() is True, and "2026".isdigit() is True.
- **d) a + b: 22** (1 pt) – Parameter $x = 4 + 5 = 9$, returns $9 \times 2 = 18 = b$. Variable *a* remains 4; $4 + 18 = 22$.
- **e) max(round(3.456, 1), min(4.2, 3.8)):** **3.8** (1 pt) – *round*(3.456, 1) = 3.5; *min*(4.2, 3.8) = 3.8; *max*(3.5, 3.8) = 3.8.

Q13. Diagnose and Correct Common Function Defects (5 points)

Solution for Q13

Code / Defect	Problem / Logic Flaw	Correction / Valid Code
<pre>def calc_area(w, h): print(w * h) ans = calc_area(5, 4) + 10</pre>	Prints area instead of returning it. Without return, yields None, raising TypeError on addition with 10. (0.8 pt)	Replace print with return: <pre>def calc_area(w, h): return w * h</pre> (0.8 pt)
<pre>def reset_counter(c): c = 0 count = 15 reset_counter(count)</pre>	Assigning <code>c = 0</code> only rebinds local parameter <code>c</code> . Does not modify caller variable <code>count</code> (remains 15). (0.8 pt)	Return new value and assign in caller: <pre>def reset_counter(): return 0 count = reset_counter()</pre> (0.8 pt)
<pre>def prepare_total(): subtotal = 100.0 prepare_total() print(subtotal)</pre>	<code>subtotal</code> is local to <code>prepare_total()</code> . Access outside raises <code>NameError: name 'subtotal' is not defined</code> . (0.8 pt)	Return value and assign to caller variable: <pre>def prepare_total(): return 100.0 subtotal = prepare_total()</pre> (1.0 pt)

Q14. Variable Scope, Name Shadowing & State Isolation (5 points)

Solution for Q14

Global Rate: 5%
 Payable Due: \$234.00
 Base Unchanged: True

Scoring Breakdown:

- Line 1 (1.5 pts):** Global Rate: 5% – Inside `apply_surcharge`, local `rate = 0.12` shadows global variable only within that scope; global `rate = 0.05` remains unchanged.
- Line 2 (2.0 pts):** Payable Due: \$234.00 – In `apply_surcharge(200.0)`, $fee = 200 \times 0.12 = 24.0$. In `process_order`, $total = 200 + 24.0 + (200 \times 0.05) = 234.0$.
- Line 3 (1.5 pts):** Base Unchanged: True – Parameter reassignments do not alter caller's `amt = 200.0`.

Q15. Function Specification, Docstrings & Stubs (5 points)

Solution for Q15

Part 1: Standard Python Docstring (2.5 points)

```
def calculate_fuel(distance, consumption_rate):
    """Computes total estimated diesel fuel consumption in liters.
    Parameters: distance (float > 0), consumption_rate (float > 0, L/100km).
    Returns: float -- Estimated fuel volume required in liters.
    """
```

Part 2: Syntactically Valid Function Stub (2.5 points)

```
def verify_token(token_str):
    # TODO: Implement cryptographic token signature validation
    return False
```

Section C: Program Design & Modular Decomposition (25 points)

Model Solution for Section C (Tasks 1 & 2) (13 points)

Task 1: Identify Functions, Parameters & Return Types (6 points, 2 pts each)

- **Function 1:** `validate_trip(distance, cargo_weight, hours_driven)` → Return Type: **bool**.
Purpose: Verifies that distance $d > 0$, cargo weight $0 < w \leq 5000.0$, and duration $t > 0$.
- **Function 2:** `compute_fuel_cost(distance, cargo_weight, fuel_price)` → Return Type: **float**.
Purpose: Determines applicable fuel consumption rate (10.0 vs 11.5 L/100km if $w > 2000$) and computes total fuel cost.
- **Function 3:** `compute_driver_pay(hours_driven, distance, is_night)` → Return Type: **float**.
Purpose: Calculates base hourly wage (\$20/hr, or \$25/hr if night shift) plus mileage allowance (\$0.30/km).

Task 2: Modular Call Hierarchy & Coordination Logic (7 points)

```
def compute_total_reimbursement(distance, cargo_weight, hours_driven, is_night, fuel_price):
    # Step 1: Input Validation Guard
    if not validate_trip(distance, cargo_weight, hours_driven):
        return 0.0

    # Step 2: Component Calculations via Dedicated Helper Functions
    fuel_cost = compute_fuel_cost(distance, cargo_weight, fuel_price)
    driver_pay = compute_driver_pay(hours_driven, distance, is_night)

    # Step 3: Subtotal and Long-Haul Bonus Application
    subtotal = fuel_cost + driver_pay
    bonus = 50.0 if distance > 400.0 else 0.0

    # Step 4: Final Total Return
    return subtotal + bonus
```

Scoring Breakdown for Task 2: 2 pts for validation guard; 2 pts for delegating to helper functions; 1.5 pts for subtotal & bonus logic; 1.5 pts for proper return type and zero side-effects.

Model Solution for Section C (Tasks 3 & 4) (12 points)**Task 3: Hand-Trace / Test Calculation for Given Scenario (6 points)**

- **Given Scenario:** $d = 300.0$ km, $w = 2500.0$ kg, $t = 5.0$ hrs, $\text{is_night} = \text{True}$, $p = \$1.50/\text{L}$.
- **Fuel Consumption Rate (1.5 pts):** Since $w = 2500.0 > 2000.0$, $\text{rate} = 10.0 \times 1.15 = 11.5$ L/100km.

- **Fuel Volume & Cost (1.5 pts):**

$$\text{Liters} = \frac{300.0 \times 11.5}{100.0} = 34.5 \text{ L} \implies \text{Fuel Cost} = 34.5 \times \$1.50 = \text{\$51.75}$$

- **Driver Pay (1.5 pts):**

$$\text{Hourly Rate} = \$20.00 \times 1.25 = \$25.00/\text{hr} \implies \text{Wage} = 5.0 \times \$25.00 = \$125.00$$

$$\text{Mileage Allowance} = 300.0 \times \$0.30 = \$90.00 \implies \text{Driver Pay} = 125.00 + 90.00 = \text{\$215.00}$$

- **Final Reimbursement Total (1.5 pts):** Distance $300.0 \leq 400.0 \implies \text{Bonus} = \0.00 .

$$\text{Total Reimbursement} = \$51.75 + \$215.00 + \$0.00 = \text{\$266.75}$$

Task 4: Software Engineering Design Decisions (6 points, 3 pts each)**a) Returning Values vs. Direct Screen Printing (3 points):**

Returning values enables functions to be modular, composable, and testable. A function that returns a value can be utilized in further calculations, automated unit test suites, GUI controls, or web APIs. A function that prints directly hardcodes presentation to standard output, produces uncapturable side effects, and prevents caller logic from reusing the result.

b) Explicit Parameters vs. Global Variables (3 points):

Passing explicit arguments and returning results maintains **referential transparency** and state isolation. Each function execution operates strictly on its own local activation record. In contrast, global variables introduce hidden dependencies, make hand-tracing difficult, create insidious side-effect bugs across functions, and violate encapsulation.

Section D: Applied Programming in Python (30 points)

Problem 1: Modular Retail Inventory & Tiered Volume Billing System (15 points)

Reference Implementation for Problem 1

```
def get_discount_rate(quantity):
    """Returns volume discount rate based on units purchased."""
    if quantity >= 100: return 0.15
    elif quantity >= 50: return 0.10
    elif quantity >= 10: return 0.05
    return 0.0

def compute_shipping(weight, is_express):
    """Computes shipping fee based on weight tier and speed."""
    base_shipping = 5.00
    if weight > 5.0:
        base_shipping += (weight - 5.0) * 1.50
    if is_express:
        base_shipping += 12.00
    return base_shipping

def compute_order_total(unit_price, quantity, weight, is_express, tax_rate=0.08):
    """Computes all invoice line items and final order payable."""
    gross = unit_price * quantity
    discount = gross * get_discount_rate(quantity)
    net = gross - discount
    shipping = 0.0 if (net >= 200.0 and not is_express) else compute_shipping(weight, is_express)
    tax = net * tax_rate
    total = net + shipping + tax
    return gross, discount, net, shipping, tax, total

def main():
    try:
        price = float(input("Enter unit price ($): "))
        qty = int(input("Enter quantity: "))
        weight = float(input("Enter total weight (kg): "))
        is_express = (input("Express delivery (yes/no)? ").strip().lower() == "yes")
        if price <= 0 or qty <= 0 or weight <= 0:
            print("Error: Price, quantity, and weight must be positive.")
            return
        gross, disc, net, ship, tax, total = compute_order_total(price, qty, weight, is_express)
        print(f"Gross Subtotal: ${gross:.2f}")
        print(f"Volume Discount ({get_discount_rate(qty)*100:.0f}%): ${disc:.2f}")
        print(f"Net Merchandise: ${net:.2f}")
        print(f"Shipping Fee: ${ship:.2f}" if ship > 0 else "Shipping Fee: Free")
        print(f"Sales Tax (8%): ${tax:.2f}")
        print(f"Total Payable: ${total:.2f}")
    except ValueError:
        print("Error: Invalid numeric input entered.")

if __name__ == "__main__":
    main()
```

Marking Rubric for Problem 1 (15 points)

- **Function `get_discount_rate` (3 points):** Correct tiered volume logic (0%, 5%, 10%, 15%) with mutually exclusive conditions and proper return statements.
- **Function `compute_shipping` (3 points):** Correct base fee (\$5.00), excess weight math above 5.0 kg (\$1.50/kg), and express surcharge (\$12.00).
- **Function `compute_order_total` (4 points):** Correct calculation of gross, discount amount, net merchandise, conditional free shipping waiver evaluation, tax, and 6-tuple return.
- **Function `main()` & Driver (3 points):** Prompting user inputs, data type conversions, validation guard, helper coordination, and receipt formatting.
- **Code Architecture & Scope (2 points):** Zero global variables, proper parameter passing, pure helper functions.

Problem 2: Student Academic Standing & Merit Scholarship Evaluator (15 points)

Reference Implementation for Problem 2

```
def validate_student_record(student_id, gpa, credits_earned):
    """Validates student ID format, GPA boundary, and non-negative credits."""
    if len(student_id) != 8 or not student_id.startswith("NEU"):
        return False
    if not student_id[3:].isdigit():
        return False
    if not (0.0 <= gpa <= 4.0) or credits_earned < 0:
        return False
    return True

def determine_standing(gpa, credits_earned):
    """Categorizes student academic standing based on GPA and credit thresholds."""
    if gpa >= 3.60 and credits_earned >= 15:
        return "Dean's List"
    elif gpa >= 2.50:
        return "Good Standing"
    elif gpa >= 2.00:
        return "Academic Warning"
    return "Academic Probation"

def calculate_scholarship(gpa, credits_earned, is_financial_aid):
    """Calculates merit, need-based, and full-time scholarship grant amounts."""
    if gpa >= 3.80:
        merit = 1200.0
    elif gpa >= 3.50:
        merit = 800.0
    elif gpa >= 3.20:
        merit = 400.0
    else:
        merit = 0.0

    need_aid = 350.0 if (is_financial_aid and gpa >= 2.50) else 0.0
    ft_grant = 150.0 if (credits_earned >= 15 and merit > 0.0) else 0.0
    return merit + need_aid + ft_grant

def main():
    student_id = input("Enter Student ID: ").strip()
    try:
        gpa = float(input("Enter Cumulative GPA (0.0 - 4.0): "))
        credits_earned = int(input("Enter Earned Credits: "))
    except ValueError:
        print("Error: GPA must be a float and credits must be an integer.")
        return

    is_aid = (input("Financial Aid Eligible (yes/no)? ").strip().lower() == "yes")

    if not validate_student_record(student_id, gpa, credits_earned):
        print("ERROR: Invalid Student Record format or values.")
        return

    standing = determine_standing(gpa, credits_earned)
    scholarship = calculate_scholarship(gpa, credits_earned, is_aid)

    print("=" * 45)
    print("ACADEMIC STANDING & SCHOLARSHIP REPORT")
    print("-" * 45)
    print(f"Student ID: {student_id} | GPA: {gpa:.2f} | Credits: {credits_earned}")
    print(f"Academic Standing: {standing}")
    print(f"Merit & Need Scholarship: ${scholarship:.2f}")
    print("=" * 45)

if __name__ == "__main__":
    main()
```

Marking Rubric for Problem 2 (15 points)

- **Function `validate_student_record` (3 points):** Accurate string validation (length 8, "NEU" prefix, 5 trailing digits) and range checks for GPA ($0.0 \leq \text{gpa} \leq 4.0$) and credits (≥ 0).
- **Function `determine_standing` (3 points):** Correct ladder logic for Dean's List ($\text{GPA} \geq 3.60 \wedge \text{credits} \geq 15$), Good Standing, Warning, and Probation with clean return values.
- **Function `calculate_scholarship` (4 points):** Correct merit tiers (\$1200, \$800, \$400), need-based supplement (\$350 if $\text{aid} \wedge \text{GPA} \geq 2.50$), and full-time course grant (\$150 if $\text{credits} \geq 15 \wedge \text{merit} > 0$).
- **Function `main()` & Report Formatting (3 points):** Safe input collection, validation call guard, helper function invocations, and cleanly formatted bordered report card.
- **Code Architecture & Modularity (2 points):** Pure helper functions, proper parameter passing, zero global mutations.

— END OF SOLUTIONS & RUBRIC —